

Zerf AI

ЭКОСИСТЕМА

ВТОРОГО МОЗГА

Долгосрочный план развития: от задачника — к связанной системе из CLI, семантического поиска, расширений и автоматизаций.

О ЧЁМ ЭТОТ ДОКУМЕНТ

Готовый поэтапный план на 12 месяцев: фундамент экосистемы (токены и общая лента), CLI в стилистике Claude Code, поиск в духе Perplexity поверх своих заметок и интернета, система плагинов как в Obsidian, автоматизации и открытый API. Сроки, стек, монетизация и критерии успеха для каждой фазы.

CLI-агент

Семантический поиск

zerf ask

Плагины и маркетплейс

Автоматизации

Открытый API

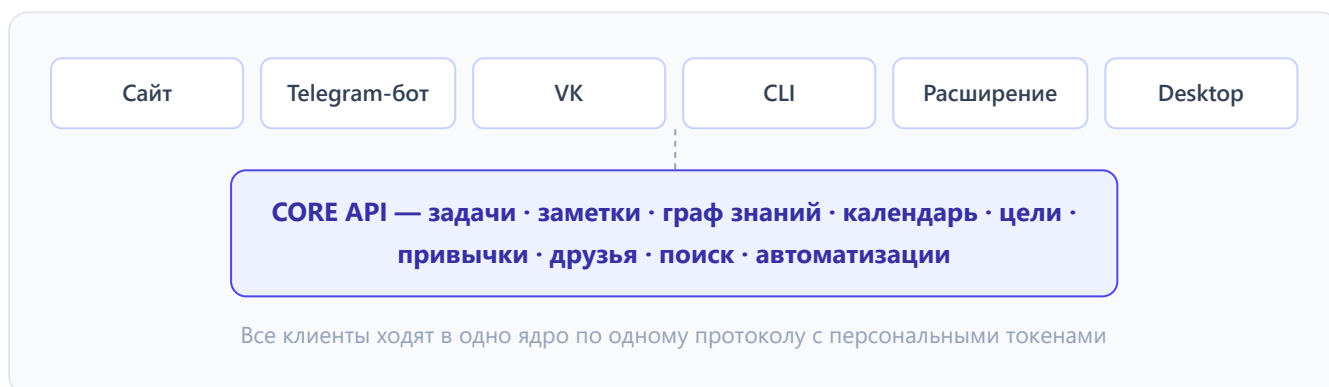
01 Содержание

2. Видение: одно ядро — много клиентов	стр. 2
3. Отправная точка: что уже готово	стр. 2
4. Решение по поиску: внутри программы, а не отдельный сервис	стр. 3
5. Дорожная карта: обзор фаз	стр. 4
6. Фаза 0 — Фундамент экосистемы	стр. 5
7. Фаза 1 — CLI в стилистике Claude Code	стр. 5–6
8. Фаза 2 — Семантический поиск и zerf ask	стр. 7–8
9. Фаза 3 — Входы и ритуалы	стр. 9
10. Фаза 4 — Система живёт сама	стр. 10
11. Фаза 5 — Открытая экосистема: API, плагины, воркспейсы	стр. 11–12
12. Система расширений в деталях (как в Obsidian)	стр. 12–13
13. Тарифная сетка экосистемы	стр. 14
14. Метрики успеха	стр. 14
15. Чего сознательно не делаем	стр. 15
16. Первые шаги — с чего начинаем	стр. 15

02 Видение: одно ядро — много клиентов

Zerf перестаёт быть «сайтом с фидами» и становится **системой второго мозга**: человек ловит мысли и контент откуда угодно, система сама организует их, и он находит всё мгновенно — в любом клиенте.

Ключевой принцип: **весь функционал пишется один раз в ядре и автоматически становится доступен во всех клиентах** — сайт, Telegram-бот, VK, CLI, браузерное расширение, desktop-компаньон. Переключение между клиентами бесшовно, потому что это одна база, одна лента активности, одни настройки.



Критерий успеха видения: заметка, созданная в терминале, через секунду видна на сайте, в боте и в расширении — и в ленте активности видно, откуда она появилась.

03 Отправная точка: что уже готово

Фундамент экосистемы уже существует — большой рефакторинг не нужен:

- **Ядро API:** ~40 эндпоинтов (задачи, заметки с папками, цели, привычки, календарь, друзья, подписки, поиск), бизнес-логика сосредоточена в `lib/backend/db.ts`.
- **Три клиента уже работают поверх ядра:** сайт, Telegram-бот, VK-колбэк.
- **Авторизация:** session-токены (100 дней), Telegram initData HMAC, VK-подписи, одноразовые login-токены — PAT добавляется поверх готовой инфраструктуры.
- **AI-слой:** Groq с пулом ключей и фолбэками, Whisper-транскрипция, Vision OCR, недельные отчёты, утренние/вечерние дайджесты.
- **Монетизация:** тарифы Free / Plus 99 ₽ / Pro 299 ₽ / Corp, YooMoney-оплаты, промокоды, реферальные счётчики.
- **Инфраструктура:** Supabase (Postgres), Vercel, cron-расписания, ICS-фид, админ-панель.

Важно: отдельный этап «выделение ядра» из плана исключён сознательно — API уже является ядром. Нужны только персональные токены, пара недостающих эндпоинтов и версионирование контрактов для внешних клиентов.

04 Решение по поиску: внутри программы, а не отдельный сервис

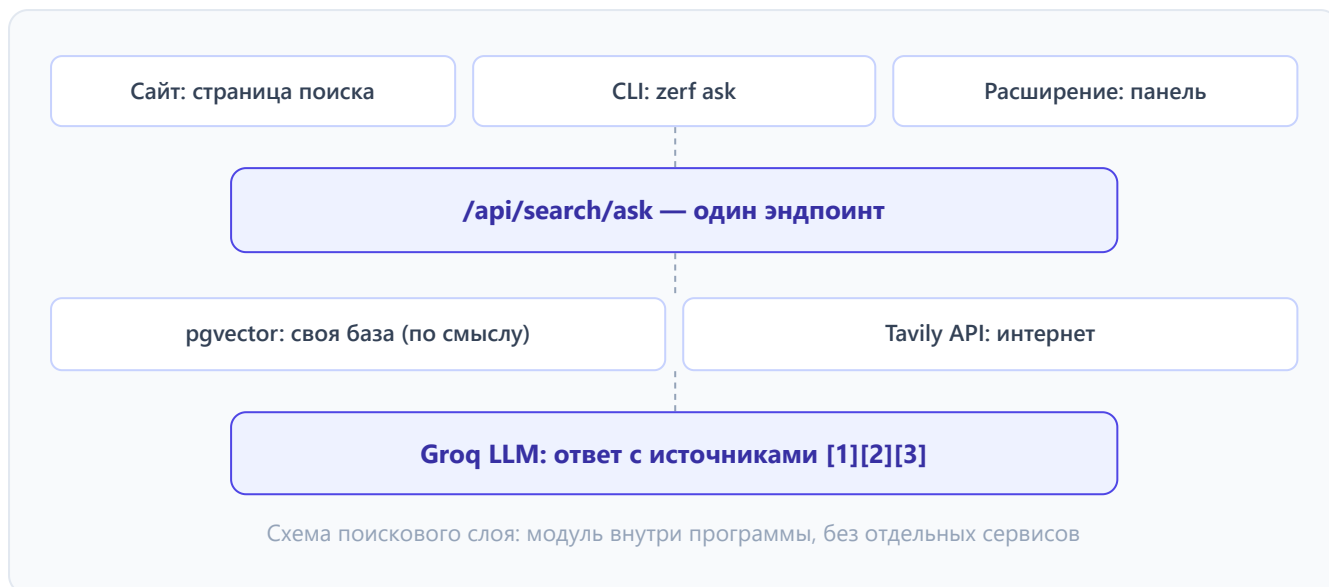
Поисковик — это **не отдельное приложение и не отдельный сервер**. Это модуль внутри существующего Next.js-бэкенда, который доступен из всех клиентов через один эндпоинт.

Почему не отдельно

- Отдельный сервис = отдельное развёртывание, мониторинг, синхронизация данных и точка отказа — на текущем масштабе это чистые издержки без выгоды.
- Данные для поиска уже лежат в нашей Supabase-базе — векторный индекс живёт в той же БД (расширение pgvector), без репликации куда-либо.
- Индексировать весь интернет не нужно и невозможно: веб-часть ответов берётся готовой через поисковый API (Tavily) — как это делает Perplexity.

Как устроено: три компонента внутри программы

Компонент	Что это	Где живёт
Векторный индекс	Embeddings заметок, задач, сохранённых статей и истории бота. Поиск по смыслу: «как я решал проблему масштабирования» находит заметку «растёт нагрузка на бота».	Таблица <code>Embedding</code> + <code>pgvector</code> в той же Supabase
Поисковый модуль	Единая функция: параллельный запрос к своему индексу и к Tavily API (веб-результаты), затем LLM (Groq) собирает ответ с источниками — «из твоих заметок» / «из интернета».	<code>lib/backend/search.ts</code> — обычный модуль бэкенда
Один эндпоинт	Все клиенты дергают один и тот же контракт. Пишем один раз — Perplexity-опыт доступен везде.	<code>/api/search/ask</code>



Вынос поиска в отдельный микросервис оправдан только при нагрузке в сотни запросов в секунду — на горизонте плана этого не будет. Архитектура модуля позволяет сделать это позже без переписывания клиентов: они общаются с эндпоинтом, а не с внутренностями.

05 Дорожная карта: обзор фаз

Шесть фаз на 12 месяцев. Каждая фаза заканчивается работающим продуктом, который можно выпускать и монетизировать, — без «больших взрывов».

Фаза	Срок	Главный результат	Тариф
0. Фундамент	недели 1–2	PAT-токены, лента активности, deep links, метрики в админке	основа для всего
1. CLI MVP	недели 2–5	Терминальный клиент с полным функционалом сайта: команды + REPL-агент	Plus
2. Поиск	месяцы 2–3	Семантический поиск по базе + zerf ask (Perplexity-слой)	Pro (лимиты ниже)
3. Входы и ритуалы	месяцы 3–5	Браузерное расширение, daily notes с шаблонами, публикация, версияность	Plus / Pro
4. Живая система	месяцы 5–9	Автоматизации, email-вход, RSS-кюратор, desktop-компаньон	Pro
5. Открытая экосистема	месяцы 9–12	Публичный API, webhooks, плагины как в Obsidian, маркетплейс, воркспейсы	Pro / Corp

06 Фаза 0 — Фундамент экосистемы

Токены, лента, мосты, метрики

1–2 недели

Цель: бэкенд готов принимать внешних клиентов безопасно, а переключение между клиентами ощущается как одна система.

Задачи:

- **РАТ-токены:** сессии с типом `cli` / `pat` и сроком 1 год. Выдача через бота: команда `/api` генерирует одноразовый код, `zerf login` активирует его. Механика login-токенов уже существует — переиспользуем.
- **Лента активности:** таблица `ActivityLog` (пользователь, действие, объект, клиент-источник, время). Даёт «магию»: «3 заметки изменены в Telegram со вчера».
- **Deep links:** `zerf open <id>` открывает объект на сайте; из бота — кнопка с готовой командой CLI. Мостики между клиентами.
- **Метрики в админке:** DAU/WAU, retention D1/D7, конверсия Free → Plus → Pro, выручка по месяцам. Без цифр дальнейшие фазы летят вслепую.
- **Недостающие эндпоинты:** поиск по заметкам (текстовый — временно), batch-операции, версия контрактов `/api/v1` для внешних клиентов.

Результат: внешние клиенты подключаются за минуты

Риски: минимальные — всё поверх готового

07 Фаза 1 — CLI в стилистике Claude Code

Терминальный клиент над всем функционалом

2–4 недели, параллельно с фазой 0

Цель: пользователь ни в чём не ограничен в терминале: задачи, заметки, цели, привычки, календарь, друзья — весь функционал сайта.

Стек: Node + TypeScript (наш стек), **ink** (React для терминала — так сделан UX Claude Code) или `@clack/prompts`, commander для аргументов. Публикация в npm как `zerf`.

Два режима:

- **Команды для скорости:** `zerf new`, `zerf find`, `zerf today`, `zerf done <query>`, `zerf habit`, `zerf cal`, `zerf inbox`, `zerf open`, `zerf sync`.
- **REPL-агент:** баннер, естественный язык → Groq function calling → инструменты (те же API-эндпоинты) → план → подтверждение Y/n → диф перед записью. Стилистика Claude Code / Gemini CLI.

Защита и права: операции-destructive требуют подтверждения; опасные (`zerf rm`) — двойного. Токен в `~/.zerf/credentials`, права по тарифу проверяет сервер — CLI не хранит тарифную логику.

Монетизация: CLI — фича Plus; первая «фича для своих»

Аудитория: разработчики — самая платящая

08 Фаза 2 — Семантический поиск и zerf ask

Поисковик, который знает тебя

месяцы 2–3

Цель: один вопрос — ответ из своих заметок и из интернета, с источниками. Главная «продающая» фишка экосистемы.

Шаг 1 — векторный индекс (2–3 недели):

- Включить расширение **pgvector** в Supabase, таблица **Embedding** (objectId, kind, vector), индекс HNSW.
- Embeddings: у Groq нет embedding-модели — берём OpenAI **text-embedding-3-small** (копейки: ~0,02 \$ за миллион токенов) или бесплатный HF Inference (пул уже умеет HF). Бэкфил всех заметок + пересчёт при изменениях (очередь через cron).

Шаг 2 — zerf ask и поиск на сайте (1–2 недели):

- **lib/backend/search.ts**: параллельно pgvector (своя база) + **Tavily API** (интернет; бесплатный тариф 1000 запросов/мес) → Groq собирает ответ с источниками.
- Эндпоинт **/api/search/ask**; клиенты: страница поиска на сайте, **zerf ask " ... "** в CLI, позже панель расширения.
- Режимы: **--web**, **--local**, по умолчанию — всё вместе.

Пример: **zerf ask "какие подходы к версионированию заметок и что я сам об этом писал?"**

→ «Из твоих заметок: ... (12.03)» + «Из интернета: CRDT, OT...» → ответ со ссылками [1][2][3].

Монетизация: Pro — безлимит; Plus — 10/день; Free — 3/день попробовать

Риск: качество склейки — тюнинг промптов 1–2 недели

09 Фаза 3 — Входы и ритуалы

Ловить откуда угодно, возвращаться каждый день

месяцы 3–5

Цель: база наполняется сама из браузера, а ежедневные ритуалы формируют привычку (retention).

Браузерное расширение (1,5–2 недели), три в одном:

- **Клиппер:** страница → в базу с автовыжимкой (Groq), автотеги, связи с похожими заметками.
- **Панель «Из твоей базы»:** при гуглении сбоку — что ты уже писал и сохранял по теме (после фазы 2 это дёшево — поиск уже есть как API).
- **Quick capture:** выделил текст → правый клик → «Сохранить в Zerf» с контекстом страницы. Manifest V3, авторизация по PAT.

Daily notes и шаблоны (1 неделя): ежедневная заметка (незавершённое вчера, расписание, рефлексия), таблица [Template](#) с переменными («книга», «встреча», «проект»), автосвязь со вчерашней заметкой.

Публикация (несколько дней): заметка/папка → публичная страница с работающими wikilinks. Рендер Markdown и граф уже есть. Аналог Obsidian Publish (\$8/мес только за это) — наша Pro-фича почти бесплатно.

Версионность заметок (1 неделя): таблица ревизий, дифы, восстановление. Ожидаемая платная фича для всех, кто пришёл из Notion.

Монетизация: клиппер — Plus; публикация и версии — Pro

Эффект: переключение поведения — «всё само падает в базу»

10 Фаза 4 — Система живёт сама

Автоматизации, email, RSS, desktop

месяцы 5–9

Цель: система не ждёт команд — она замечает, предлагает и делает сама.

Движок автоматизаций (2–3 недели): таблица правил «триггер → условие → действие».

Триггеры: заметка создана, задача просрочена, тег поставлен, AI заметил дубликат. Действия: создать задачу, написать в бот, связать заметки, предложить МОС. Обобщение захардкоженных cron-правил; cron-инфраструктура уже есть.

Email-вход (1–2 недели): письмо на `inbox@домен` (Cloudflare Email Routing → webhook, почти бесплатно) → LLM-классификация: задача / статья / расписание → правильное место. Пересылка без думания — сильная привычка.

RSS + AI-кюратор (1–2 недели): news-fetcher уже тянет внешние источники — раскрываем для пользователей: свои ленты → выжимки → папка «На почитать», автосвязь с заметками по темам.

Desktop-компаньон (3–4 недели, Tauri): хоткей → всплывающая панель: быстрый capture, поиск (включая ask), today, уведомления. Переиспользует API-клиент и UI-код CLI. Raycast-стиль — главное окно в экосистему на десктопе.

Монетизация: автоматизации и кюратор — Pro; desktop — Plus

Эффект: «система помогает, а не ждёт» — главное отличие от конкурентов

11 Фаза 5 — Открытая экосистема

API, плагины, маркетплейс, воркспейсы

месяцы 9–12

Цель: пользователи сами достраивают продукт друг для друга — как сообщество Obsidian. Это самый дешёвый двигатель развития.

Workspaces (2–3 недели): личное / работа / проект, переключение контекста, поиск по всем воркспейсам, шаринг целого воркспейса. Corp становится настоящим командным тарифом с высоким чеком, а не «безлимитом для одного». Аккуратная миграция схемы.

Публичный API + webhooks (1–2 недели): PAT уже есть — добавляем документацию, лимиты по тарифу, вебхуки на события («заметка создана», «задача завершена») в n8n / Zapier / свой сервер. Привлекает техническую аудиторию и создаёт комьюнити.

Плагины и маркетплейс — подробно в следующем разделе: три уровня открытости, манифест с правами, каталог с модерацией.

Монетизация: плагины бесплатны (рост ценности), воркспейсы и API-лимиты — Pro / Corp

12 Система расширений в деталях (как в Obsidian)

Принцип: **продукт — платформа, комьюнити — двигатель фич**. Obsidian вырос не на своих фичах, а на тысячах плагинов сообщества. Повторяем этот путь тремя уровнями открытости — от простого к сложному.

Уровень 1 — Публичный API (основа, фаза 5)

Без API плагины невозможны. Всё, что построено на PAT-токенах и версионированных эндпоинтах. Разработчик может написать скрипт/бота/интеграцию уже на этом уровне.

Уровень 2 — CLI-плагины (npm-пакеты, сразу после CLI)

- Формат: npm-пакеты `zerf-plugin-*`; установка `zerf plugin install <name>`; команды плагина появляются в CLI как родные.
- Плагин получает объект API-клиента с токеном и правами текущего пользователя — как Obsidian-плагин получает `app`.
- Исполняется локально на машине пользователя — сандбокс не нужен (та же модель доверия, что у npm/Obsidian).
- Примеры, которые напишем сами для затравки: экспорт дневника в Markdown-архив, интеграция с git, статистика в терминале, импорт из Todoist.

Уровень 3 — Веб-плагины (сандбокс, фаза 5)

- Манифест: имя, версия, права (`notes:read` , `tasks:write` , `network`) — пользователь видит и одобряет права при установке.
- Исполнение в изолированном `iframe` / `Web Worker`; связь с приложением через `postMessage`-мост к ограниченному набору операций — произвольный доступ к DOM и данным невозможен.
- UI-слоты: команда, панель, вид (карточка на дашборде), пункт в меню заметки. Рендер через безопасный поднабор HTML/CSS.
- **Серверного исполнения чужого кода нет** — только клиентские плагины плюс доверенные официальные автоматизации. Это снимает 90% проблем безопасности.

Маркетплейс

- Каталог: в админке (модерация) и на сайте (установка в один клик); для CLI — реестр npm-пакетов.
- Версионирование `semver`, счётчики установок, отзывы позже; плагины бесплатны на первом этапе.
- В дальнейшем — `rev-share` для авторов (процент Pro-подписок, приведённых плагином) — сильный стимул для комьюнити.

Порядок внедрения: API → CLI-плагины (дёшево, локальное доверие) → веб-плагины (сандбокс) → маркетплейс. Не пытаться сделать всё сразу: сначала 5–7 собственных плагинов-примеров, потом открытие для сообщества.

13 Тарифная сетка экосистемы

Тариф	Что входит	Лимиты ИИ
Free	Сайт + боты, базовые лимиты задач/напоминаний, текстовый поиск	3 запроса ask/день — попробовать
Plus · 99 ₽/мес	Всё из Free + CLI , клиппер, безлимит заметок/Siri/целей, daily notes, desktop-компаньон	10 ask/день, автовыжимки статей
Pro · 299 ₽/мес	Всё из Plus + ask безлимит , автоматизации, публикация, версионность, RSS-кюратор, повышенные API-лимиты	безлимит, приоритетная очередь
Corp	Всё из Pro + воркспейсы и командный доступ, общие проекты, позже SSO	командные пулы лимитов

14 Метрики успеха

Оцениваем не количество фич, а три цифры (все три — в админке с фазы 0):

Метрика	Что означает	Какая фаза на неё влияет
DAU / WAU	Ежедневная привычка пользоваться системой	CLI, daily notes, дайджесты
Retention D7 / D30	Люди возвращаются после первой недели	ask, автоматизации, расширение
Конверсия Free → Plus → Pro	Готовность платить за экосистему	ask-проба (5+ запросов → покупка), CLI, плагины

Правило: каждая фаза после релиза наблюдает за своей метрикой 2–4 недели; следующая фаза стартует только если метрика предыдущей не падает.

15 Чего сознательно не делаем

- **Конкурент Google / свой индекс интернета** — это корпорация с миллиардными затратами, а не фича. Наша ниша: поиск, который знает пользователя, поверх чужого индекса (Tavily).
- **Большой рефакторинг «выделение ядра» до старта CLI** — API уже ядро; двигаемся инкрементально.

- **Серверное исполнение плагинов пользователей** — только клиентские плагины и доверенные автоматизации.
- **Canvas/whiteboards, свой браузер, нативные мобильные приложения** — дорого, слабый эффект на текущем этапе; PWA закрывает мобильный кейс.
- **Фазы 3–5 раньше 1–2** — клиппер без семантического поиска просто копилка статей; автоматизации без стабильного ядра — источник багов.

16 Первые шаги — с чего начинаем

Неделя 1: PAT-токены + `zerf login` через код из бота; таблица `ActivityLog`; метрики DAU/WAU в админку.

Неделя 2: каркас CLI на ink: авторизация, `today`, `new`, `find`, `done`, `sync`, `open`.

Недели 3–4: REPL-агент (Groq function calling поверх API), привычки/календарь/друзья в CLI, публикация пакета в npm.

Параллельно: pgvector + embeddings-бэкфилг — фундамент поиска готов к фазе 2.